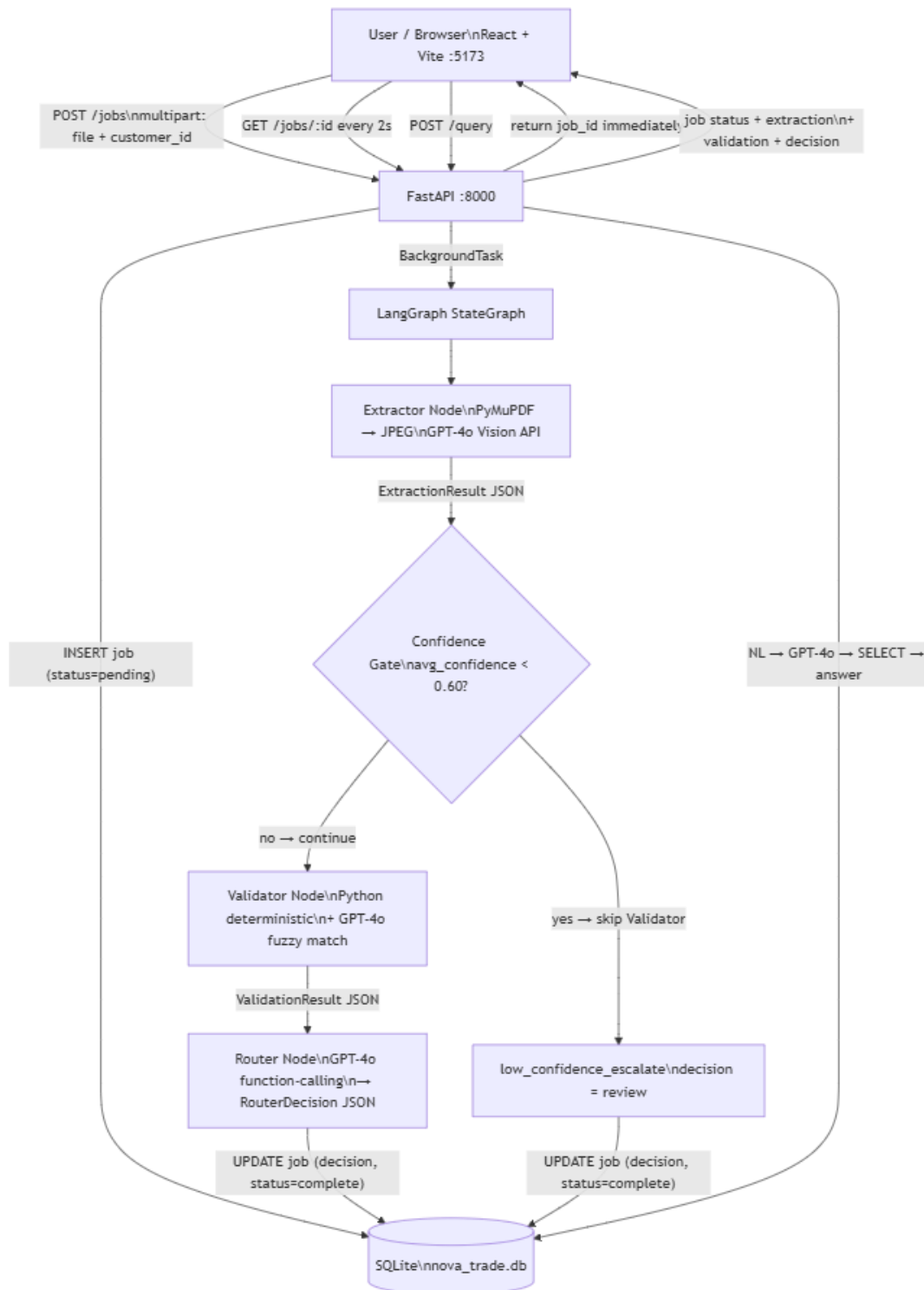


Nova Trade Document Pipeline — Technical Write-Up

1. Architecture



Upload returns the `job_id` in $<100\text{ms}$; LangGraph runs the pipeline as a background task. All agent handoffs are typed Pydantic models stored as JSON in SQLite — no shared in-memory state. Conditional edges enforce the `confidence_gate` → `validator` ordering so low-confidence docs never hit the Validator.

2. Three Real Failure Modes from Testing

Failure 1 — GPT-4o wraps fuzzy-match JSON in markdown fences.

The Validator calls GPT-4o for consignee name fuzzy matching. GPT-4o returned the JSON object wrapped in ````json```` code fences, causing `json.loads()` to raise `JSONDecodeError` and silently crash every fuzzy-match field. Fix: strip fence characters before parsing. Lesson: GPT-4o adds markdown formatting even when the prompt says "return JSON only" — always strip fences regardless of whether you use function-calling.

Failure 2 — Multi-value HS codes always mismatch on multi-line-item invoices.

ELECPARTS_003's approve doc has three line items; GPT-4o extracted all three HS codes as one comma-separated string "8541.10, 8541.21, 8541.29". The exact-match check compared that whole string against single-value entries in `allowed_values` — always a mismatch. Fix: split by comma and check each code individually. Lesson: `allowed_values` validation is a set-membership check, not a string equality check — design for multi-value extraction from day one.

Failure 3 — Verbose UN/LOCODE descriptions break exact-match port validation.

Sample PDFs were generated with descriptive port text (CNSHA (Shanghai Yangshan)). GPT-4o faithfully extracted the parenthetical; the rule expected clean CNSHA. Every port check failed even on otherwise-correct documents. Fix: regenerated all 7 sample PDFs with clean UN/LOCODE codes. Lesson: sample-data quality determines test validity — if fixtures have verbose strings while rules expect codes, no amount of logic fixes will produce a passing test.

3. Observability for 50-Customer Production

The POC already has the foundation: every job carries a `job_id` UUID through all stages, status is updated at each hop (pending → extracting → validating → routing → complete/failed), full agent outputs are stored as JSON in SQLite, and the `/query` NL endpoint enables ad-hoc investigation without a separate tool.

To scale to 50 customers, add:

- **`job_id`-correlated structured logging** on every log line across all agents and the API layer
- **Prometheus metrics** — `nova_jobs_total{status, customer_id}` and `nova_agent_latency_seconds{agent}` at `/metrics`; alert on >10% failed over 10 minutes or p95 latency >90s
- **Per-customer dashboard** — auto-resolve rate, human override rate, top mismatched fields
- **OpenTelemetry traces** — one span per pipeline stage with `job_id` as trace attribute, exportable to Grafana/Jaeger

4. Cost Per Document (Back-of-Envelope)

Using Azure OpenAI GPT-4o pricing as of mid-2025 (~0.0025/1Kinputtokens,0.01/1K output tokens):

Stage	What runs	Tokens	Cost
Extractor	GPT-4o vision: 4 JPEG images (~500 tokens/image) + 1.5K text prompt + 600 output	~3.6K in / 600 out	~\$0.015
Validator	GPT-4o fuzzy match (consignee only; all other fields are free Python)	~300 in / 100 out	~\$0.002
Router	GPT-4o: ValidationResult + draft email generation	~800 in / 400 out	~\$0.006
Total			~\$0.023/doc

Main risk: multi-page PDFs (4 pages → ~\$0.05/doc) and runaway retries (capped at 9 LLM calls max via MAX_RETRIES_PER_AGENT=2 × 3 agents). Hard controls: max_pages=4 in the PDF renderer, MAX_TOKENS_PER_JOB=20000, LANGGRAPH_RECURSION_LIMIT=15.

5. Latency Breakdown

Stage	Observed	Bottleneck
PDF → JPEG (PyMuPDF)	~200ms	CPU-bound, negligible
Extractor (GPT-4o vision)	15–25s	Slowest hop — image tokens take time
Validator (Python)	<100ms	Free for exact/regex fields
Validator (GPT-4o fuzzy)	1–3s	Only called when fuzzy-match fields present
Router (GPT-4o + draft email)	3–6s	Email drafting is the longest text generation
Total p50	~20–30s	Dominated by Extractor

To reduce the Extractor bottleneck: enable GPT-4o prompt caching (system prompt is identical across all jobs); lower JPEG zoom from 1.5× to 1.0× (cuts image tokens ~40% with no legibility loss); parallelize per-page extraction calls for multi-page docs.

6. What I'd Do Differently with a Week

- **Postgres instead of SQLite.** At 50 customers with concurrent uploads, SQLite's write serialisation becomes a bottleneck. Swapping aiosqlite for asyncpg in connection.py gives native JSONB indexing, connection pooling, and truly concurrent writes — code change is small.
- **Email trigger.** Documents in real GoComet ops arrive as email attachments. An IMAP watcher or email-gateway webhook auto-submitting to POST /jobs would make the pipeline fully autonomous — no manual upload step.
- **Cross-document consistency validation.** Currently each doc (BoL, CI, Packing List) is validated in isolation. A post-validation step that cross-checks consignee, HS codes, and gross weight across all three documents in a shipment set would catch a whole class of discrepancies that per-document rules miss.
- **OpenTelemetry distributed traces.** Structured logging with job_id is a start, but OTel spans per pipeline stage (with model, token count, and latency as attributes) would enable single-pane-of-glass visibility in Grafana/Jaeger without log mining.
- **Golden dataset eval harness.** 50 manually verified documents as fixtures, scored on extraction F1 and decision accuracy via pytest --eval. Makes prompt changes and model upgrades safe to ship — regression score before merge.